

ABC Fitness - Complete Interview Preparation Guide for Girish

This guide is customized specifically for your upcoming AI/Software Engineering interviews targeting ABC Fitness Hyderabad. It combines your internship experience, full-stack engineering knowledge, AI/ML understanding, DevOps skills, system design preparation, backend concepts, and company-specific preparation.

1. Understanding ABC Fitness

ABC Fitness is a global SaaS company operating in the fitness technology industry. The company serves millions of users and thousands of gyms worldwide through scalable software platforms.

Their focus areas include:

- AI-powered fitness products
- scalable backend systems
- SaaS platform engineering
- cloud-native architectures
- AI-assisted product development
- intelligent scheduling systems
- AI workout recommendation systems
- engineering productivity through AI tooling

The Hyderabad engineering team contributes across:

- engineering
- AI product development
- backend systems
- operations
- R&D; workflows

Interview expectations:

- practical engineering thinking
- backend/system design understanding
- AI integration understanding
- communication
- scalable product thinking
- production-focused development

The company appears to value engineers who can combine:
AI + software engineering + practical product execution.

2. Why You Are a Strong Fit

Your profile aligns strongly because you already have:

- backend API development experience
- async systems experience
- AI/LLM integration exposure
- production-oriented engineering mindset
- system design understanding
- cloud + DevOps exposure

- reinforcement learning exposure
- GPU tooling experience
- scalable backend workflows

Your internship work shows:

- ability to understand large codebases
- research-oriented engineering
- practical AI integration
- reliability engineering
- backend scalability thinking

This matches the type of engineering mindset ABC Fitness appears to value.

3. Refined Self Introduction

Hi, I'm Girish, a Data Science undergraduate with strong interest in AI-driven software engineering, backend systems, and scalable product development.

Recently, I worked as an AI/Software Development Intern at DrugParadigm, where I contributed to a reinforcement learning-based molecular generation system inspired by AstraZeneca's PROTAC Invent framework.

My work focused on backend API development, GPU-accelerated computational workflows, and reliability engineering. I integrated tools like ROSHAMBO and OpenMM into the reinforcement learning scoring pipeline and implemented asynchronous APIs with retry and fallback mechanisms for long-running computations.

Apart from that, I've built multiple full-stack and AI-powered projects such as:

- EmotiLearn, an adaptive learning platform for dyslexic children,
- FixMyHyd, an AI-powered civic issue reporting platform,
- and Nivasa, a full-stack booking platform.

I also have experience with cloud and DevOps tooling including AWS, Docker, Kubernetes, Terraform, Redis, and CI/CD pipelines.

Alongside development, I actively practice DSA and have solved 350+ LeetCode problems. I enjoy building scalable systems where AI and backend engineering come together to solve practical real-world problems.

4. Internship Deep Technical Questions

Q1. Explain the RL workflow in your internship project.

Answer:

The system used a pretrained LSTM-based molecular generative model trained on ChEMBL datasets. The user provides ligand SMILES strings, 3D structures, scoring constraints, and optional docking files.

During reinforcement learning iterations, the agent model generates candidate molecules. These are evaluated using the reinvent-scoring pipeline, which computes reward scores using shape similarity, conformer generation, docking, ADMET properties, and energy minimization.

The loss is computed using the difference between prior and agent negative log likelihood values, and the agent model updates iteratively to generate higher-scoring molecules.

Q2. Why did you replace licensed tools?

Answer:

The original implementation depended on licensed computational chemistry tools like ROCS, OMEGA, and Schrödinger. I researched open-source alternatives to reduce dependency costs while maintaining acceptable performance and reliability.

Q3. Why ROSHAMBO?

Answer:

ROSHAMBO provided GPU-accelerated shape similarity computation and integrated well with RDKit workflows. It also enabled parallel computation for improved throughput.

Q4. Why OpenMM?

Answer:

OpenMM provided a powerful open-source alternative for molecular energy minimization workflows and reduced dependency on expensive licensed software.

Q5. Explain retry and fallback mechanisms.

Answer:

Some computational tasks were long-running or unstable. Retry mechanisms handled transient failures automatically, while fallback mechanisms ensured alternate workflows were available if computations failed or timed out.

Q6. Biggest challenge?

Answer:

Understanding the architecture of a large research-oriented codebase and integrating GPU/Linux-based workflows into existing pipelines while maintaining reliability and compatibility.

5. Backend & Full Stack Questions

Q1. What are query optimization techniques?

Answer:

- Proper indexing
- avoiding SELECT *
- pagination
- optimized joins
- query caching
- denormalization when needed
- analyzing execution plans

Q2. What is indexing?

Answer:

Indexes improve query performance by reducing full table scans and enabling faster row lookups.

Q3. SQL vs NoSQL?

Answer:

SQL databases are relational and ideal for transactions and structured data. NoSQL databases are flexible and better for high scalability and rapidly changing schemas.

Q4. When should you prefer SQL?

Answer:

When strong consistency, joins, transactions, and relational integrity are required.

Q5. When should you prefer NoSQL?

Answer:

When scalability, flexible schemas, and distributed architectures are more important.

Q6. Explain caching.

Answer:

Caching stores frequently accessed data temporarily to reduce repeated expensive computations and improve response times.

Q7. Different caching strategies?

Answer:

- Redis caching
- CDN caching
- browser caching
- reverse proxy caching
- database query caching
- API response caching

Q8. Why Redis?

Answer:

Redis is extremely fast because it is in-memory and is useful for caching, rate limiting, session storage, and task queues.

Q9. What is rate limiting?

Answer:

Rate limiting restricts request frequency to prevent abuse and improve stability.

Q10. What is idempotency?

Answer:

An operation is idempotent if repeated execution produces the same result.

Q11. Explain JWT authentication.

Answer:

JWT provides stateless authentication by securely transmitting encoded user claims between client and server.

6. DevOps & Cloud Questions

Q1. What is Docker?

Answer:

Docker packages applications and dependencies into lightweight containers for consistent deployment across environments.

Q2. Docker vs Virtual Machine?

Answer:

Containers share the host OS kernel and are lightweight, while VMs run separate operating systems and consume more resources.

Q3. What is Kubernetes?

Answer:

Kubernetes orchestrates containerized applications, managing scaling, deployment, recovery, and networking.

Q4. Why Terraform?

Answer:

Terraform enables Infrastructure as Code for reproducible, version-controlled cloud infrastructure management.

Q5. Explain your AWS experience.

Answer:

I have worked with AWS services such as EC2, S3, Lambda, and SNS. I also used Terraform for infrastructure automation and deployment workflows.

Q6. What is CI/CD?

Answer:

CI/CD automates building, testing, and deployment processes to improve software delivery speed and reliability.

Q7. What is horizontal scaling?

Answer:

Adding more servers to distribute load.

Q8. What is vertical scaling?

Answer:

Increasing resources such as CPU or RAM on a single server.

7. Python + AI Questions

Q1. Difference between multithreading and multiprocessing?

Answer:

Multithreading shares memory and is useful for I/O-bound tasks, while multiprocessing uses separate processes and is useful for CPU-bound tasks.

Q2. What is async programming?

Answer:

Async programming enables non-blocking execution and improves concurrency for I/O-heavy workflows.

Q3. What is reinforcement learning?

Answer:

Reinforcement learning is a learning paradigm where an agent learns optimal behavior through rewards and penalties.

Q4. Explain transformers briefly.

Answer:

Transformers use attention mechanisms to process sequential data efficiently and are widely used in LLMs and NLP systems.

Q5. What is hyperparameter tuning?

Answer:

Hyperparameter tuning optimizes model parameters such as learning rate, batch size, and architecture settings to improve model performance.

Q6. What is Optuna?

Answer:

Optuna is a hyperparameter optimization framework that supports efficient search strategies like Bayesian optimization.

Q7. What is TensorBoard used for?

Answer:

TensorBoard helps monitor model training metrics, loss curves, and experiment tracking.

8. SQL Coding Questions

Second Highest Salary:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Nth Highest Salary:

```
SELECT DISTINCT salary
FROM employees e1
WHERE N-1 = (
SELECT COUNT(DISTINCT salary)
FROM employees e2
WHERE e2.salary > e1.salary
);
```

Find duplicate emails:

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Top 3 salaries department-wise:

```
SELECT *
FROM (
SELECT department, salary,
DENSE_RANK() OVER(PARTITION BY department ORDER BY salary DESC) as rnk
FROM employees
) t
WHERE rnk <= 3;
```

9. ABC Fitness Product Improvement Questions

Q1. How would you improve ABC Fitness products?

Strong Answer:

I would focus on combining personalization, scalability, and AI-driven user engagement.

Some ideas:

- AI workout personalization using user fitness history and wearable data
- intelligent scheduling optimization for trainers and gyms
- AI-generated recovery recommendations
- conversational AI fitness assistants
- predictive churn analysis for gym memberships
- AI-driven nutrition guidance integration
- real-time performance analytics dashboards

From an engineering perspective, I would also focus on:

- scalable backend architectures
- async event-driven systems
- caching optimization
- cloud-native deployments
- observability and monitoring
- AI workflow reliability

Q2. What new AI idea would you propose?

Answer:

An AI-powered adaptive fitness coach that dynamically adjusts workouts based on:

- fatigue levels
- wearable data
- recovery patterns
- user engagement
- historical progress

The system could combine recommendation systems, LLM-based conversational interfaces, and predictive analytics.

Q3. How would you improve developer productivity internally?

Answer:

I would explore:

- AI-assisted debugging workflows
- internal developer copilots
- automated documentation generation
- AI-powered test generation
- intelligent code review systems
- reusable internal platform tooling

10. Behavioral Questions

Q1. Tell me about a difficult challenge.

Answer:

Understanding a large research-oriented AI codebase involving reinforcement learning pipelines, GPU workflows, and external computational tools was challenging. I approached it systematically by tracing workflows, experimenting locally, validating outputs, and gradually understanding architecture components.

Q2. Why should we hire you?

Answer:

I combine backend engineering, AI integration, cloud tooling, system design understanding, and strong problem-solving skills. I enjoy understanding systems deeply and building reliable, scalable applications.

Q3. What are your strengths?

Answer:

Independent learning, backend engineering, problem-solving, persistence, and ability to understand large systems.

Q4. What is your weakness?

Answer:

Earlier, I tended to over-explain technical details. Over time, I learned to adapt communication depth based on audience and business context.

11. Final Interview Strategy

1. Start high-level first.
2. Then go technical gradually.
3. Never dump all details immediately.
4. Explain architecture before implementation.
5. During coding rounds:
 - clarify problem
 - explain brute force
 - optimize
 - discuss complexity
 - dry run
6. Speak calmly and slowly.
7. Show ownership in projects.
8. Focus on practical engineering thinking.
9. If you don't know something:
"I haven't implemented that directly yet, but based on my understanding..."
10. Show curiosity and product thinking, not only coding skills.